MECHANISM AI

End-to-End Architecture & Working Flow

A practical implementation blueprint for a constraint-driven mechanical design agent: from a natural-language prompt to simulation, independent validation, iterative repair, and an evidence-backed result.

Hackathon MVP	A planar four-bar linkage design assistant
Core principle	The AI proposes; an independent evaluator decides whether a candidate passes.
Primary simulation engine	MuJoCo, with an early feasibility spike for the closed-loop linkage
Application stack	React + TypeScript + Vite, FastAPI + Pydantic, Meta model API, MuJoCo, NumPy/SciPy
Deployment	Docker; JSON evidence artifacts initially; SQLite only if needed

Document purpose

This document describes how the parts work together, what each component owns, what data passes between them, and the order in which to build the MVP. It is an implementation plan, not a claim that the complete system or sample results already exist.

Version 1.0 • 27 October 2026

1. Product goal and demo promise

Mechanism AI turns a mechanical motion requirement into candidate linkage designs, runs them through a simulator, evaluates them against fixed constraints, and attempts bounded repairs when a candidate fails.

The differentiator is not simply generating a drawing. It is the closed loop: specify → generate → simulate → measure → repair → report. The evaluator is separate from the language model and applies the same acceptance criteria to every candidate.

Example live-demo prompt

“Design a compact four-bar linkage that produces approximately 100 mm of near-horizontal movement, fits within a 200 mm footprint, completes its motion cycle, and avoids collisions.”

The demo should make the validation visible. Show an initial candidate failing a measurable constraint, the exact metric that failed, a revised candidate, and the final result passing the same tests. If no candidate passes, report that honestly and show the best candidate with unresolved failures.

MVP boundary

- Support one mechanism family only: planar four-bar linkage.
- Use structured parameters rather than model-generated executable code.
- Validate kinematics and geometric constraints; do not claim structural, fatigue, materials, manufacturing, or real-world safety certification.
- Cap the repair loop (for example, three candidates) so the demo is predictable and bounded.

A successful result means it passed the implemented simulation checks and stated assumptions—not that it is ready to manufacture or safe to deploy.

2. System architecture at a glance

Layer	Responsibility	Technology
User interface	Prompt, run status, animated mechanism, candidate comparison, metrics and evidence report	React + TypeScript + Vite; SVG first, Three.js only if needed
API/orchestration	Validate requests, create run IDs, coordinate workflow, publish progress	FastAPI + Pydantic; REST + Server-Sent Events (SSE)
AI adapter	Normalize requirements, propose parameter candidates, diagnose failures and suggest bounded revisions	Meta model API behind a small provider adapter
Mechanism compiler	Convert validated parameters into a simulation model	Python schema + MJCF template/compiler
Physics simulation	Run mechanism motion and collect states, poses, and solver/contact information	MuJoCo
Independent evaluator	Measure trajectory, travel, footprint, cycle completion, collision checks and simulation validity	Python + NumPy/SciPy + domain-specific checks
Evidence storage	Save prompt, requirements, parameters, settings, traces, scores and repair history	JSON artifacts first; SQLite if needed
Runtime	Package and run the app reproducibly	Docker

Architecture rule: the model never writes or changes the acceptance rules. The evaluator owns pass/fail.

3. End-to-end flow, starting from the user prompt

Step 1 — User submits a requirement

The user enters a natural-language brief in the React UI. The browser sends it to the API and receives a run identifier. The UI begins listening for progress events and renders each event as the run advances.

```
POST /api/v1/design-runs
{
  "prompt": "Design a compact four-bar linkage...",
  "options": { "max_candidates": 3 }
}
```

Step 2 — FastAPI validates and starts the run

FastAPI checks request shape, input length, allowed mechanism type, and resource limits. It creates a run record with a unique run_id and starts the workflow. For the first MVP, keep orchestration inside the FastAPI service; add Redis/Celery only if long-running jobs, concurrency, or recovery make it necessary.

Step 3 — Meta model converts language into a strict brief

The AI adapter asks the model to return structured requirements, not prose alone. Pydantic validates the result. Separate explicit user requirements from assumptions. If a missing value materially changes the design, ask a clarification question; otherwise use a documented default.

```
{
  "mechanism_type": "four_bar_linkage",
  "units": "mm",
  "target_motion": {
    "type": "near_horizontal",
    "travel_distance": 100,
    "trajectory_tolerance": 5
  },
  "constraints": {
    "max_footprint": 200,
    "complete_motion_cycle": true,
    "avoid_collisions": true
  }
}
```

The 5 mm trajectory tolerance in this example is an illustrative default, not a value supplied in the sample prompt. The UI/report should label such assumptions clearly.

Step 4 — The model proposes a candidate parameter set

The model returns a JSON candidate containing linkage lengths, pivot/layout parameters and input motion. The application validates types, units, ranges, and allowed fields. Invalid output is rejected or retried within a strict limit. No model-generated Python, shell commands, or arbitrary executable code is run.

```
{
  "mechanism_type": "four_bar_linkage",
  "parameters": {
    "ground_length": 100,
    "crank_length": 35,
    "coupler_length": 110,
    "rocker_length": 85
  },
  "input_motion": {
    "start_angle_deg": 0,
    "end_angle_deg": 360
  }
}
```

These numbers are illustrative and are not a verified valid mechanism.

Step 5 — The mechanism compiler creates the simulation model

The compiler transforms the validated schema into MuJoCo MJCF: bodies, joints, geometry, actuator, coordinate conventions, and any obstacles used by the test. Keep the compiler deterministic: the same parameters and compiler version should produce the same model structure.

Important early feasibility check

A four-bar linkage is a closed kinematic loop. Confirm that the chosen MuJoCo representation handles loop closure correctly and stably before building the rest of the product. If closed-loop modelling is awkward or unreliable, build a small custom kinematic solver as a reference implementation and use it to test the geometry and evaluator while resolving the MuJoCo representation. Do not silently substitute a visually plausible animation for validated simulation.

3. End-to-end flow (continued)

Step 6 — MuJoCo runs the motion

The simulation runner loads the compiled model, applies the input motion, advances simulation steps, and records the state at bounded intervals. It collects joint states, body poses, output-point coordinates, solver status and collision/contact information available from the chosen model. The runner should return a structured result or a specific failure—not an unhandled crash.

Step 7 — The independent evaluator scores the candidate

The evaluator compares measured simulation outputs with the original normalized requirements. It should be deterministic and versioned. The same acceptance criteria apply to every candidate, including those proposed after a repair.

Check	Example measurement	Pass/fail intent
Travel distance	Output-point displacement in mm	Within the required range
Trajectory quality	Deviation from target path or target direction	Error below tolerance
Footprint	Bounding dimensions of the mechanism in mm	No larger than the specified maximum
Motion completion	Required input range completed without invalid state/solver failure	Full requested cycle completed
Collision checks	Geometry/contact checks for defined bodies and obstacles	No prohibited collision under the configured test
Simulation integrity	Finite coordinates, bounded values, successful run	No numerical failure or invalid output

Collision detection must be explicitly designed and tested. A simulation engine's contact output does not automatically prove every intended clearance rule is satisfied.

Illustrative evaluator output

```
{
  "candidate_id": "candidate_001",
  "status": "failed",
  "metrics": {
    "travel_distance_mm": 94,
    "footprint_mm": 185,
    "trajectory_error_mm": 8.2
  },
  "failed_constraints": ["trajectory_tolerance"]
}
```

The values above are examples only; they are not real measurements from an implemented system.

Step 8 — The repair loop uses failure evidence

If a candidate fails, the orchestrator sends the model the candidate parameters, measured failure metrics, failed constraints, allowed parameter ranges, and remaining attempt count. The model proposes a new parameter set. The application validates it, compiles it, simulates it, and runs the same evaluator again.

- Use a hard limit such as three candidates total for the demo.
- Keep all prior candidates and their test results for the comparison view.
- Do not relax constraints automatically just to obtain a pass.
- If no candidate passes, return the best candidate plus explicit unresolved failures.

Step 9 — Save an evidence bundle

Each run should be reproducible enough for debugging and credible demo claims. Save the original prompt, normalized brief, assumptions, candidate parameters, model/provider metadata where available, compiler version, MuJoCo settings, simulation trace, evaluator version, metrics, repair history and final status.

Step 10 — The UI renders the actual results

The browser consumes SSE progress events, then requests the final candidate and trajectory artifacts. The mechanism animation should be derived from coordinates produced by the simulation/kinematics pipeline. Show the target path and actual path, dimensions, metric checks, failed constraints and candidate history. Do not animate a hard-coded success sequence disconnected from the computation.

4. API contract and event flow

Recommended first endpoints

Endpoint	Purpose
POST /api/v1/design-runs	Validate prompt/options, create run, return run_id and initial status
GET /api/v1/design-runs/{run_id}	Return overall status, stage, current candidate and final summary if available
GET /api/v1/design-runs/{run_id}/events	SSE stream for progress events
GET /api/v1/design-runs/{run_id}/candidates	List candidate parameters, scores and failures
GET /api/v1/health	Health/readiness check
GET /api/v1/design-runs/{run_id}/artifacts/{name}	Optional endpoint for trajectory/report files

Example event sequence

```
event: run_started
data: {"run_id":"run_123","stage":"parse_requirements"}

event: requirements_parsed
data: {"run_id":"run_123","mechanism_type":"four_bar_linkage"}

event: candidate_generated
data: {"run_id":"run_123","candidate_id":"candidate_001"}

event: simulation_completed
data: {"run_id":"run_123","candidate_id":"candidate_001"}

event: evaluation_completed
data: {"run_id":"run_123","status":"failed","failed_constraints":["trajectory_tolerance"]}

event: repair_started
data: {"run_id":"run_123","attempt":2,"max_attempts":3}

event: run_completed
data: {"run_id":"run_123","status":"passed"}
```

Treat event payloads as status updates, not the source of truth for the final report. The final GET endpoints should return persisted results. Include error events for invalid model output, simulation failure, timeouts and exhausted repair attempts.

Core data objects

- **DesignRun:** run_id, status, stage, original_prompt, created_at, error_code, final_candidate_id.
- **NormalizedBrief:** mechanism_type, units, target_motion, hard_constraints, assumptions, evaluator_config.
- **Candidate:** candidate_id, parameters, input_motion, generated_at, attempt_number.
- **Evaluation:** status, metrics, failed_constraints, evaluator_version, diagnostic_notes.
- **EvidenceBundle:** prompt, brief, candidates, simulation traces, settings, metrics, repair history and final outcome.

5. Suggested repository layout

```
mechanism-ai/
■■■ apps/
■   ■■■ api/
■   ■   ■■■ app/
■   ■       ■■■ api/
■   ■       ■   ■■■ routes_design_runs.py
■   ■       ■   ■■■ routes_artifacts.py
■   ■       ■■■ orchestration/
■   ■       ■   ■■■ design_run.py
```

```
■   ■       ■   ■■■ repair_loop.py
■   ■       ■■■ ai/
■   ■       ■   ■■■ meta_client.py
■   ■       ■   ■■■ requirements_parser.py
■   ■       ■   ■■■ candidate_parser.py
■   ■       ■■■ mechanism/
■   ■       ■   ■■■ schemas.py
■   ■       ■   ■■■ compiler.py
■   ■       ■   ■■■ mjcf_templates/
■   ■       ■■■ simulation/
■   ■       ■   ■■■ mujoco_runner.py
■   ■       ■   ■■■ trajectory.py
■   ■       ■■■ evaluation/
■   ■       ■   ■■■ evaluator.py
■   ■       ■   ■■■ geometry_checks.py
■   ■       ■   ■■■ metrics.py
■   ■       ■■■ persistence/
■   ■           ■■■ artifacts.py
■   ■■■ web/
■       ■■■ src/
■           ■■■ features/
■               ■■■ design-input/
■               ■■■ simulation/
■               ■■■ validation-report/
■               ■■■ candidate-history/
■■■ tests/
■   ■■■ test_schemas.py
■   ■■■ test_evaluator.py
■   ■■■ test_geometry.py
■   ■■■ test_simulation.py
■■■ docker-compose.yml
■■■ README.md
```

Keep the evaluator and mechanism geometry tests usable without calling the language model. That enables repeatable tests, faster debugging, and a benchmark of known passing and failing cases.

6. Reliability, safety and evidence

Trust boundaries

- **Untrusted AI output:** parse into typed schemas; reject unknown fields and out-of-range values.
- **No arbitrary code execution:** AI output is data only. Do not run generated Python, shell, or MJCF fragments without deterministic compiler control.
- **Resource bounds:** cap candidate count, simulation steps, solver iterations, model tokens, retries, input size and wall-clock time.
- **Secret management:** keep model API keys on the server; never ship keys to the browser.
- **Consistent criteria:** evaluator thresholds are fixed for the run and cannot be changed by the model.
- **Failure honesty:** timeouts, invalid geometry and no-pass outcomes must be visible, not silently converted to success.
- **Reproducibility:** store evaluator/compiler versions and all relevant simulation settings.

What the MVP can and cannot claim

Can claim after tests prove it	Must not claim from this MVP alone
A candidate passed the named kinematic/geometric checks under the saved configuration.	The mechanism is structurally strong, fatigue-safe, manufacturable, or certified.
The simulated output trajectory met a stated tolerance.	The physical device will match the simulation exactly.
The configured collision tests found no prohibited collisions.	All real-world collision or pinch hazards have been eliminated.
The repair loop improved a measured metric or found a passing candidate.	The AI has proven the design is universally optimal.

7. Build order: prove the hard part first

Do not start with a polished frontend. The riskiest technical item is a trustworthy four-bar representation and evaluator. Establish that before investing in the model loop and visual polish.

Milestone	Work	Exit criterion
1. Geometry/evaluator spike	Implement a deterministic four-bar kinematic reference; define units, output point, path metrics and footprint.	Known valid/invalid cases are classified correctly by tests.
2. MuJoCo feasibility	Compile and run a closed-loop representation; compare its trajectory against the reference solver.	Stable, repeatable trajectory within a defined comparison tolerance—or document a fallback plan.
3. API + artifacts	FastAPI run lifecycle, Pydantic schemas, JSON persistence and error states.	A run can be started, polled, and inspected without AI.
4. UI visualization	Prompt form, mechanism animation, target-vs-actual path, metric panel and status stream.	UI displays real trace data and truthful status.
5. Meta model integration	Parse requirements and produce candidate JSON through a provider adapter.	Model output validates against schema; invalid outputs fail safely.
6. Repair loop	Feed measured failure evidence back to model; cap attempts.	Repair can improve a benchmark case without changing the evaluator.
7. Demo hardening	Benchmark, error handling, rehearsals, fallback recording and deployment.	Five consecutive rehearsals finish without a critical failure.

8. Nine-day hackathon schedule

Date	Focus	Deliverable
Oct 27	Lock schema; test Meta API access; define benchmark cases.	Clear scope + passing/failing reference cases
Oct 28	Evaluator, geometry, trajectory and footprint checks.	Deterministic evaluator tests
Oct 29	MuJoCo closed-loop proof; build first visual trace.	Verified simulation or explicit fallback decision
Oct 30	React interface and API run lifecycle.	Prompt-to-status-to-trajectory path
Oct 31	Meta model parsing/candidate generation.	Structured, validated candidate output
Nov 1	Repair loop and candidate comparison.	Bounded generate-test-repair cycle
Nov 2	Evidence report, UX polish and user feedback.	Demo story and benchmark results
Nov 3	Hardening, failure cases and rehearsal.	Backup video + stable build
Nov 4	Freeze, verify submission and submit early.	Final submission

9. Acceptance targets and demo script

Suggested engineering targets (goals, not existing results)

- At least 90% correct pass/fail classification on a fixed, manually reviewed evaluator benchmark.
- At least 70% repair success on deliberately repairable test cases within three total candidates.
- Zero known false passes on the benchmark; a false pass is more damaging than a reported failure.
- Aim for a typical demo run under 60 seconds if the simulator/model latency makes this achievable.
- Five consecutive end-to-end rehearsals with no critical failure.
- Five viewers can explain what passed and why; three relevant users express concrete interest or identify a real workflow they would try.

Suggested live demo sequence

Time	What the audience sees	Message
0–10 sec	Enter the plain-language requirement.	The user specifies motion and constraints, not CAD commands.
10–20 sec	Requirements become a visible structured brief; assumptions are labelled.	The AI translates intent into testable requirements.
20–35 sec	Candidate 1 moves; evaluator marks one metric as failed.	The system does not confuse plausible geometry with success.
35–50 sec	Candidate 2 is generated and simulated; metrics update.	The agent repairs based on measured evidence.
50–60 sec	Pass/fail report and candidate comparison appear.	Every claim is tied to a saved test result.

Include an intentionally impossible or contradictory prompt in a separate short demo/test. The correct behavior is to say no valid candidate was found within the search budget and identify the conflict—not to fake a pass.

10. Key implementation decisions

Decision	Recommendation	Reason
Mechanism scope	Four-bar linkage only	Depth and reliability beat broad but shallow mechanism coverage.
Physics engine	MuJoCo first; do not integrate MuJoCo and PyBullet together for the MVP	One simulation contract keeps implementation and debugging bounded.
Closed-loop modelling	Validate immediately; use a custom kinematic reference solver as a cross-check	The four-bar closed loop is a high-risk technical dependency.
AI output	Strict JSON parameter schema only	Avoids unsafe execution and makes results inspectable.
Workflow engine	Simple FastAPI orchestration initially	Avoid premature complexity; add queue infrastructure only when needed.
Persistence	JSON evidence bundles, optional SQLite	Enough for one-user/demo-scale runs and reproducibility.
Visualization	SVG first; add Three.js only if it materially improves clarity	Fast, controllable, and adequate for a planar linkage demo.

11. Final checklist

- The evaluator can classify fixed known cases without an LLM call.
- The MuJoCo model and reference kinematics agree within a defined tolerance.
- Every model response is schema-validated and bounded.
- The same acceptance criteria are applied to every candidate.
- The UI animation uses actual simulation/kinematic output data.
- Every result includes assumptions, metrics, failed checks and version metadata.
- No-pass and simulation-error outcomes are clear and honest.
- The demo is rehearsed with a backup recording and the submission is prepared before the deadline.

Bottom line

Build the independent evaluator and prove the four-bar simulation first. Then connect the Meta model as a candidate proposer and repair assistant. Finish with a UI that makes the evidence visible. That sequence creates the strongest technical story: an AI that is judged by an external test, learns from measured failure, and reports what it can and cannot prove.

Technical note: All sample JSON values and metric values in this document are illustrative examples. Confirm the chosen MuJoCo closed-loop formulation, Meta API availability, latency and licensing in the actual development environment before relying on them.